

ATM Simulator

Multi-User Edition with Admin Panel

Project Report

Course	Python Essentials (VITyarthi)
Student Name	Rohit Maity
Registration No.	26BCE11118
Submission Date	25 September 2026

1. Introduction

Automated Teller Machines are one of the most familiar examples of a small, self-contained transactional system: a user authenticates, performs a bounded set of operations against a stored balance, and every action must be validated, logged, and kept consistent even when the user makes mistakes. This project recreates that system as a console-based Python application, extended beyond a single hard-coded account into a proper multi-user system with persistent storage and an administrative role

The goal was not just to reproduce ATM menus, but to apply core software engineering concepts covered in the course: modular design, input validation, secure credential handling, persistent storage, and automated testing — all within a scope appropriate for a single-file console application turned into a properly structured Python package.

2. Problem Statement

A typical beginner ATM script models one hard-coded account with an in-memory balance: all data is lost when the program exits, and there is no way to support more than one user or to recover a locked-out account. This project addresses that gap by building a multi-user ATM simulator that supports account registration, secure PIN-based login with lockout protection, everyday banking operations, and a separate admin role for account recovery — with all data persisted between runs using a local JSON file.

Scope

In scope:

- Multi-user account registration and login via a console interface
- Core banking operations: balance check, deposit, withdrawal Per-account, timestamped transaction history
- PIN security: hashed storage, 3-attempt lockout, self-service PIN change
- An admin role that can list users, force a password reset, unblock a locked account, and inspect any user's history
- Persistent storage using a local JSON file (no external database server required)

Out of scope:

- A graphical or web user interface (console-only)
- Real banking integrations, multi-currency support, or interest calculations
- Concurrent/multi-terminal access to the same data file
- Production-grade encryption at rest (PINs are hashed, but the JSON file itself is not encrypted)

Target Users

- Bank customers (regular users) — create an account, log in with their account number and PIN, and manage their own balance.
- Bank administrator — a single admin role responsible for resetting forgotten passwords and unblocking accounts after failed login attempts.

3. Functional Requirements

The system is organised into three major functional modules, each with a clear input/output structure and a logical workflow:

Module 1 - Account Management

- Register a new account: input = name + chosen PIN; output = auto-generated account number and a starting balance of Rs. 5,000
- Log in to an existing account: input = account number + PIN; output = access to the user menu, or a rejection with remaining-attempts feedback
- Change PIN: input = current PIN + new PIN (entered twice); output = updated stored credential

Module 2 - Banking Transactions

- Check balance: output = current balance, formatted as currency
- Deposit funds: input = amount; output = updated balance, rejects non-numeric or non-positive input
- Withdraw funds: input = amount; output = updated balance, rejects amounts greater than the current balance
- View transaction history: output = a timestamped log of every deposit, withdrawal, and account event

Module 3 – Admin Panel

- List all users: output = every account's number, name, balance, block status, and last login
- Force a password reset: input = target account number; effect = user is prompted to set a new PIN on next login, bypassing the old one
- Unblock a locked account: input = target account number; effect = clears the block flag and resets the failed-attempt counter
- View any user's history: input = target account number; output = that account's full transaction log

Logical Workflow

Welcome Menu → Existing / New Account → Authentication → User Menu or Admin Menu → selected action performed and saved → Exit returns to the Welcome Menu (see the Workflow Diagram, Section 7).

4. Non-Functional Requirements

Requirement	Requirement How it is addressed
Security	PINs are never stored or compared in plain text — only their SHA-256 hash is kept. Accounts lock after 3 failed attempts
Reliability	All state changes (deposit, withdraw, PIN change, admin actions) are written to disk immediately, so a crash never loses a completed transaction.
Usability	Clear numbered menus, explicit error messages (e.g. remaining PIN attempts, insufficient balance), and confirmation prompts for PIN changes
Maintainability	The codebase is split into 7 single-responsibility modules instead of one script, so a change to one concern (e.g. storage format) does not ripple through the others
Error Handling Strategy	Every user input path (PIN, amount, menu choice) is validated with a re-prompt or a clear rejection message instead of an unhandled exception
Resource Efficiency	The JSON database is small and read/written only on demand; no background processes or polling.

5. System Architecture

The system follows a simple layered design: a console front-end (main.py) drives three logic modules (auth, user_ops, admin_ops), all of which share common helpers (utils, constants) and go through a single persistence layer (database.py) that is the only module allowed to touch the storage file. This keeps each module independently testable and means the storage format (JSON today) could be replaced with a real database later by editing only database.py.

ATM System - Architecture

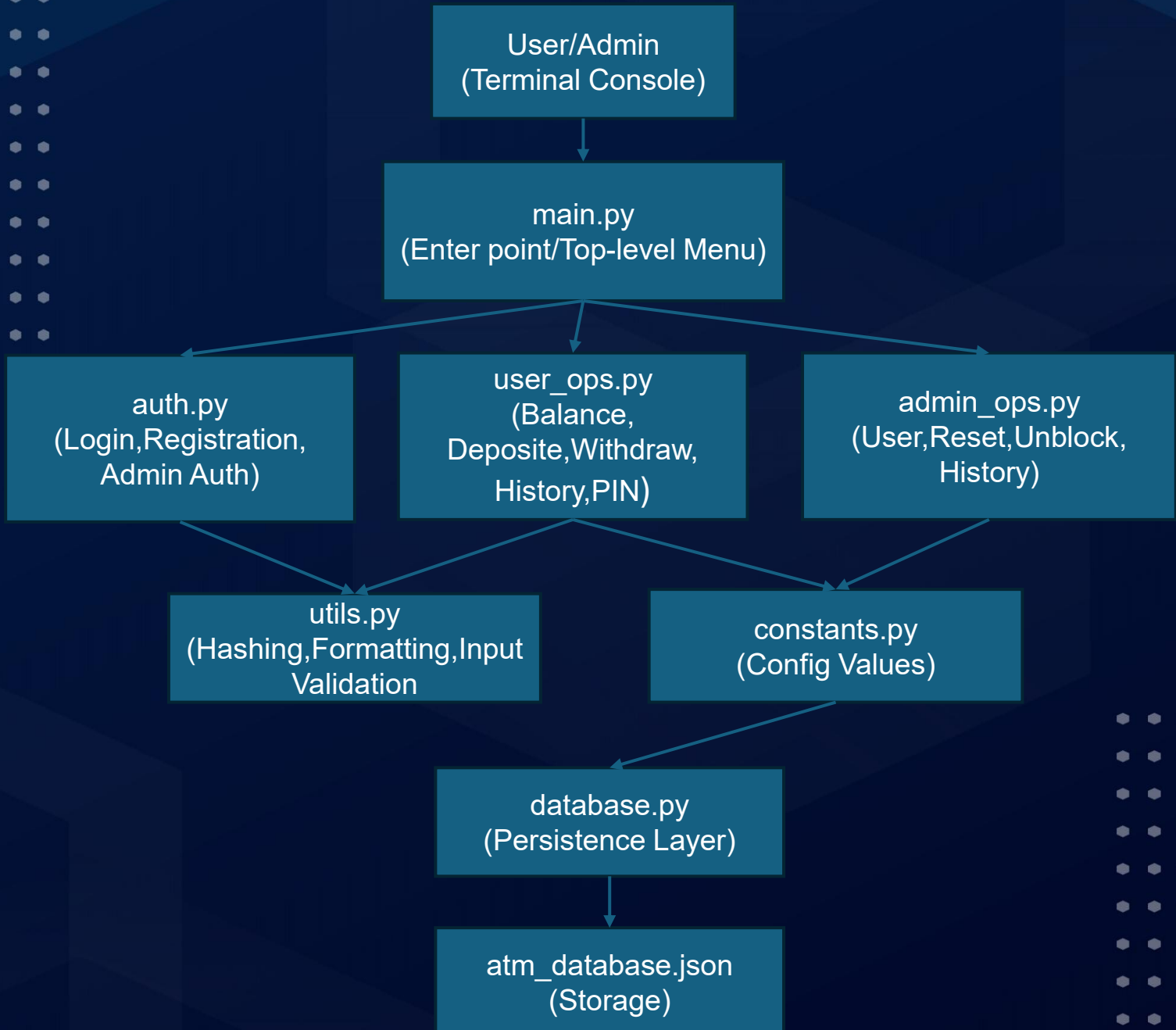


Figure 1 - System Architecture Diagram

6. Design Diagrams

6.1 System Architecture

Two actors interact with the system: the regular User and the Admin. Their available operations do not overlap, except that both must authenticate first

Use Case Diagrams

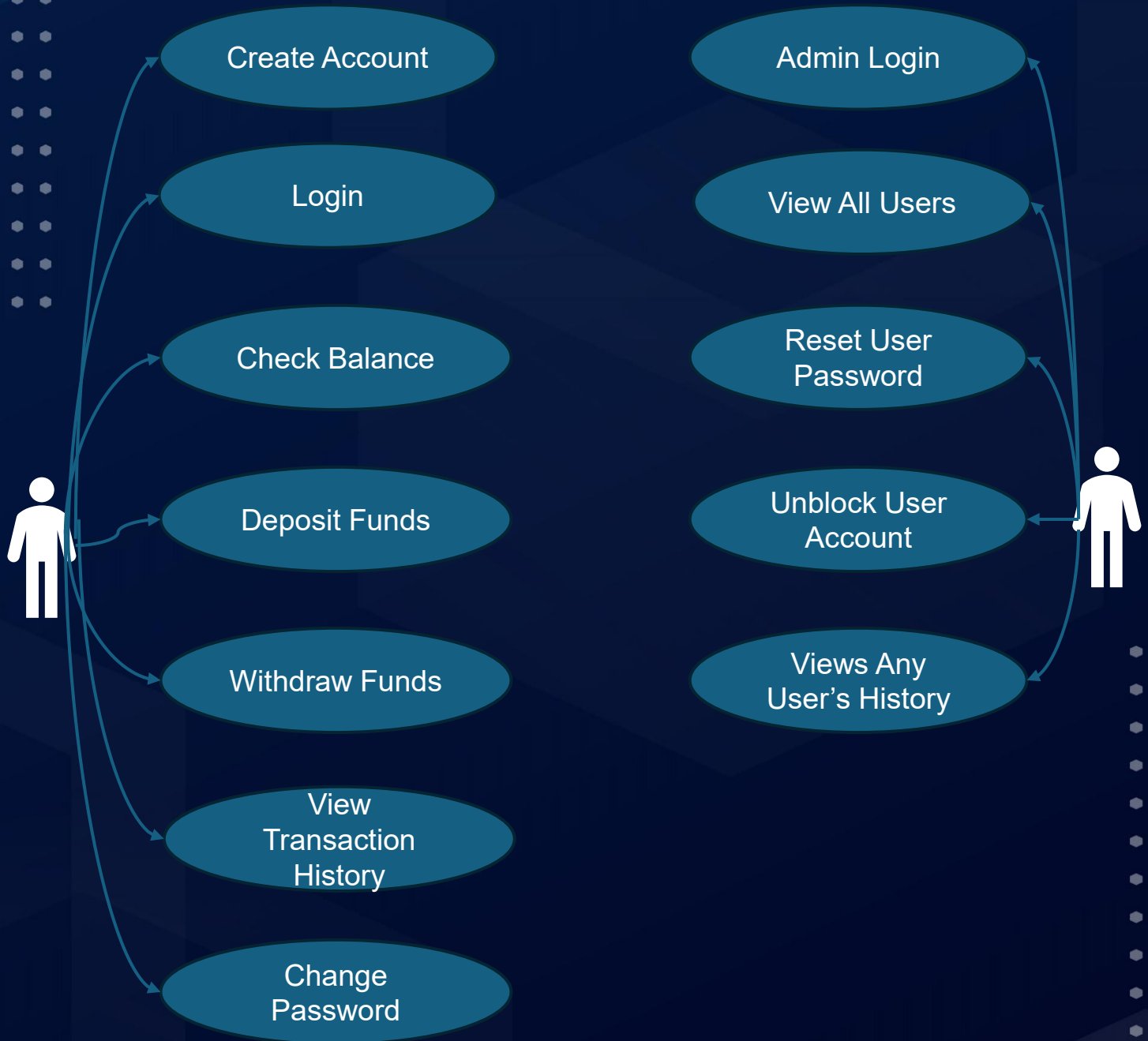


Figure 2 – Use Case Diagram

6.2 System Architecture

The top-level control flow, from program start through account access to menu-driven actions and back to the welcome screen on exit.

ATM WorkFlow

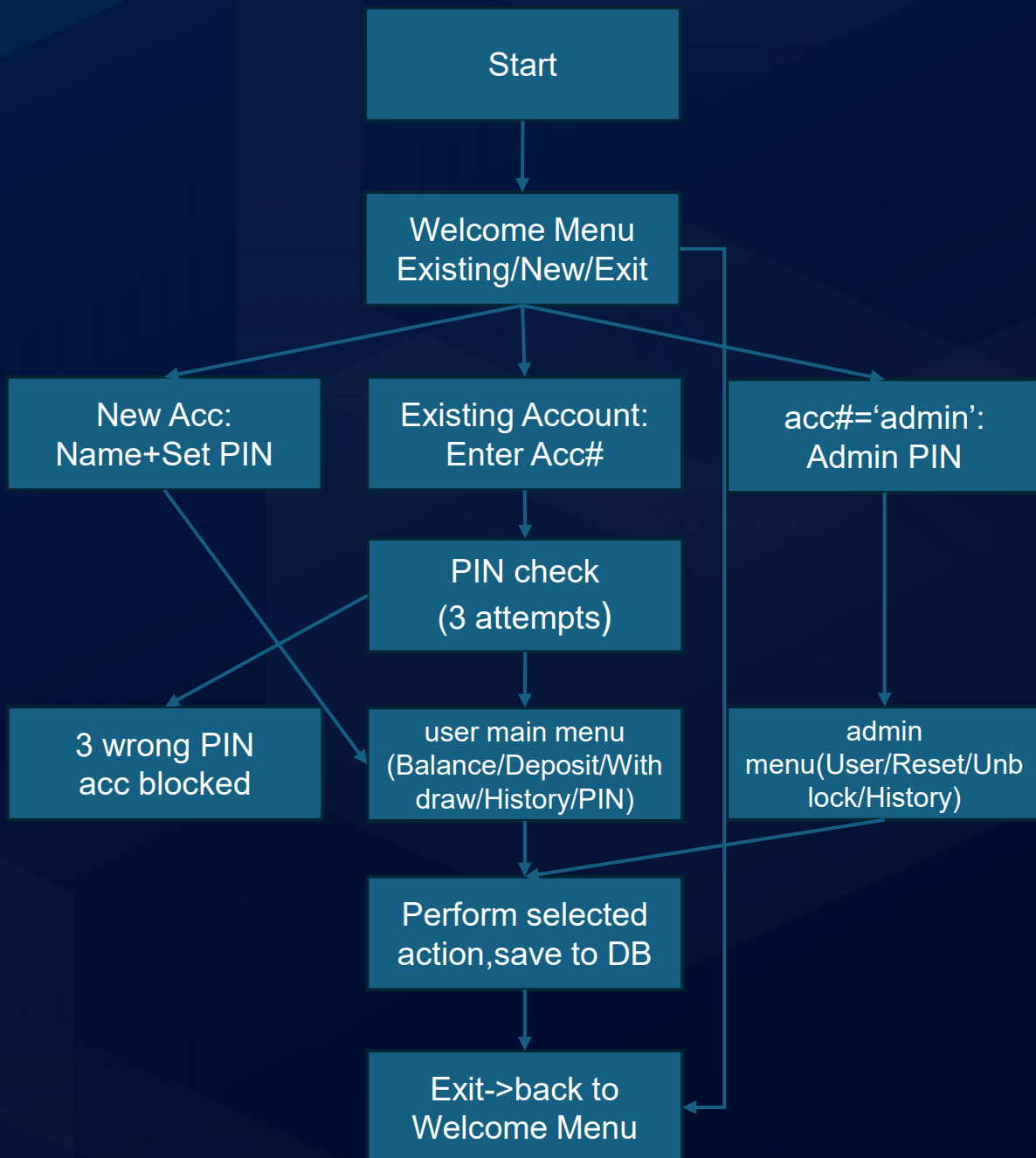


Figure 3 – Process Workflow Diagram

6.3 Sequence Diagram

A successful login walks through five participants: the User, the top-level main.py loop, the auth module, the database module, and the JSON file itself.

Existing User Login

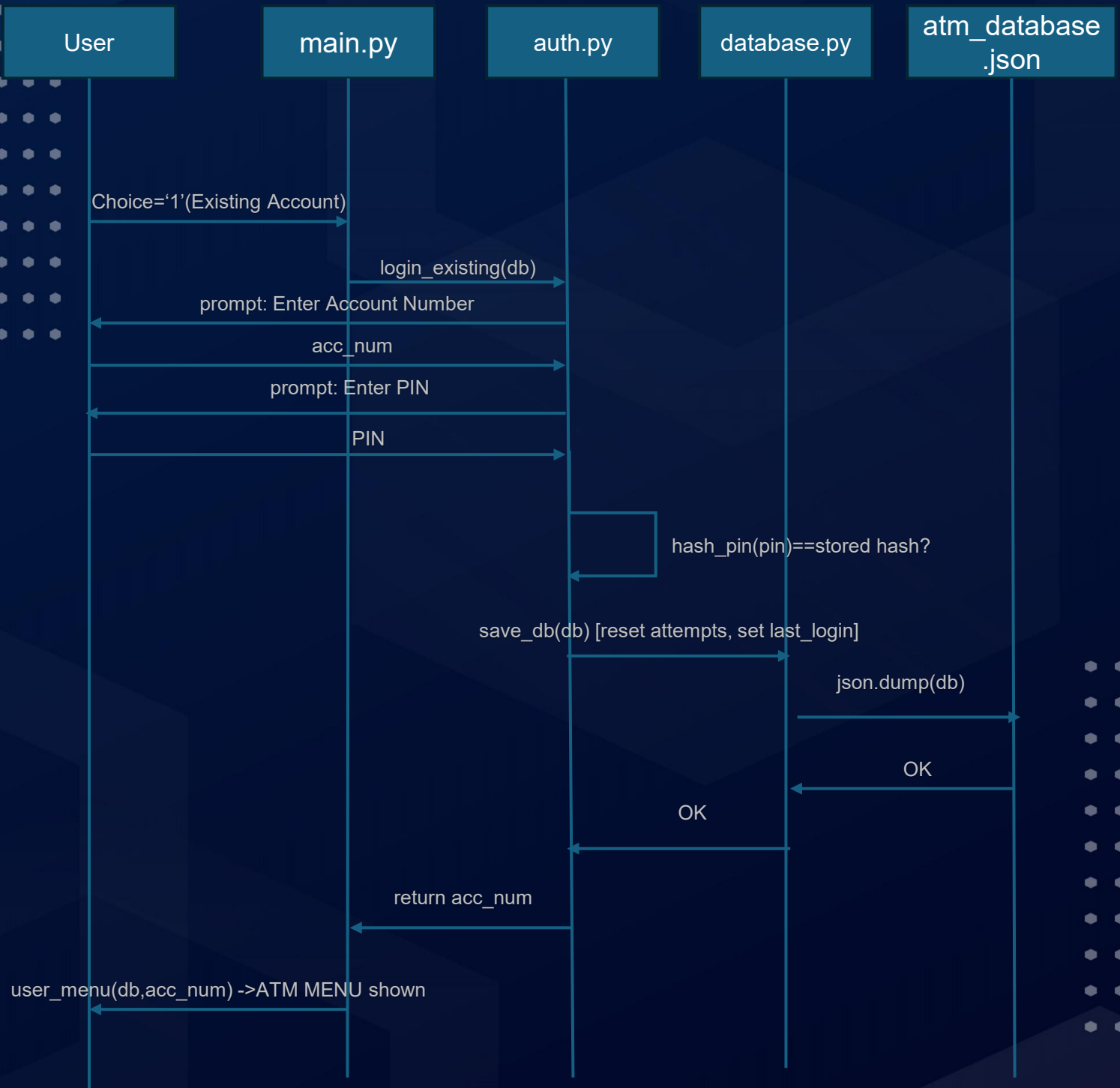


Figure 4 – Sequence Diagram (Existing User Login)

6.4 Class/Component Diagram

The implementation is function-based rather than built from Python classes, but the data it manages maps cleanly onto three logical entities shown below.

Class Diagram(conceptual Data Model)

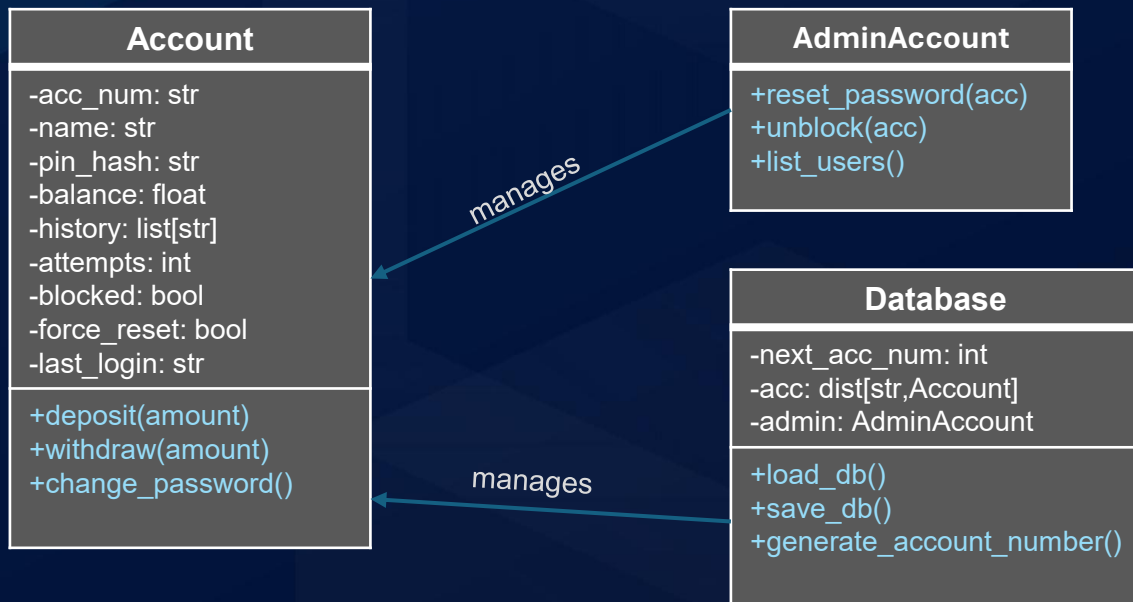


Figure 5 – Conceptual Class Diagram

6.5 ER Diagram/Schema Design

Because storage is a JSON document rather than a relational database, the "foreign key"(FK) from an account to its history is really just a nested list, as reflected below.

ER Diagram-atm_database.json

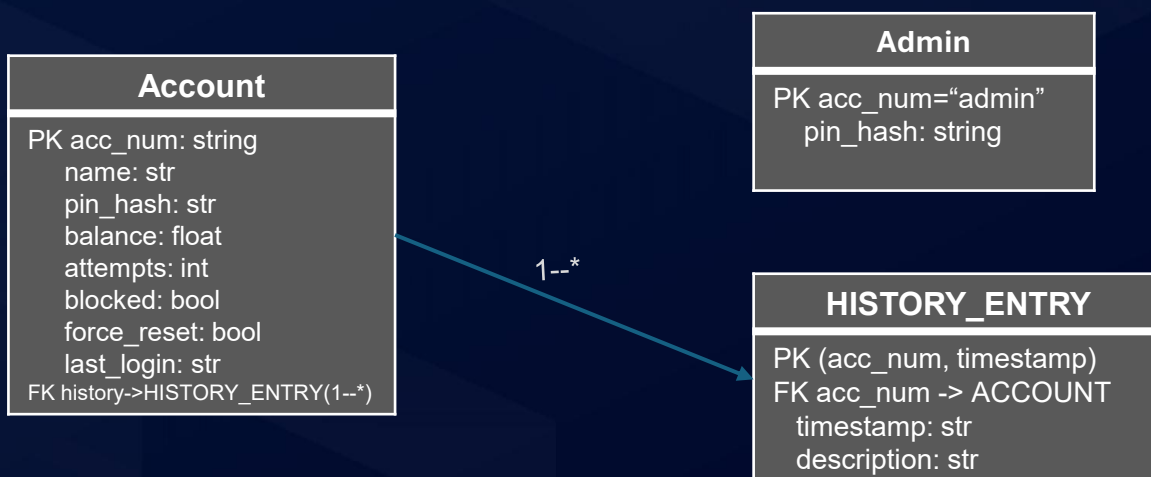


Figure 6 – ER Diagram of atm_database.json

7. Design Decisions & Rationale

Why JSON instead of a real database?

The project scope is a single-terminal console app with no concurrent access requirement, so a real DBMS (SQLite/PostgreSQL) would add setup overhead without adding value. A JSON file gives human-readable persistence with zero external dependencies, while `database.py` isolates that choice behind a small function interface (`load_db/save_db`) that could be swapped out later

Why functions instead of classes?

The domain objects (an account, an admin) are plain data with no internal invariants that need encapsulation beyond what validation functions already provide. A function-per-operation style keeps each unit small, easy to unit test in isolation (see Section 11), and easy to follow for a console workflow that is itself procedural (menu → action → save)

Why hash PINs instead of encrypting them?

PINs only ever need to be verified, never recovered and displayed back to a user — that is exactly what a one-way hash (SHA-256) is for. Encryption would imply the system could decrypt and show the PIN again, which is neither needed nor desirable from a security standpoint.

Why admin-forced reset instead of a security question?

Security questions are commonly guessable or forgotten themselves. Routing recovery through a human admin (as most real institutions do for a forgotten PIN) is simpler to implement correctly and matches how the assignment described the desired admin workflow

8. Implementation Details

The project is implemented in pure Python 3 (standard library only for the runtime code) as seven modules plus a test package:

Module	Responsibility
<code>constants.py</code>	Central configuration: file paths, starting balance, max PIN attempts, default admin PIN.
<code>utils.py</code>	Pure helper functions: PIN hashing, timestamping, currency formatting, validated input readers
<code>database.py</code>	The only module that reads/writes <code>atm_database.json</code> ; exposes <code>load_db</code> , <code>save_db</code> , <code>get_account</code> , <code>generate_account_number</code>
<code>auth.py</code>	Account creation, existing-account login with lockout logic, and admin authentication
<code>user_ops.py</code>	The logged-in user's menu and its actions: balance, deposit, withdraw, history, change password.
<code>admin_ops.py</code>	The admin menu and its actions: list users, force password reset, unblock account, view any user's history
<code>main.py</code>	Entry point; wires the modules together into the top-level loop.

Key Implementation Choices

- PIN hashing: `hashlib.sha256(pin.encode()).hexdigest()` — the same helper is used everywhere a PIN is set or checked, so there is exactly one place credential handling can go wrong.
- Lockout: an attempts counter on the account record increments on each wrong PIN and is reset to 0 on any successful login or admin unblock.
- Admin-forced reset: a `force_reset` boolean flag on the account; when set, the next login skips the PIN check entirely and asks the user to set a new one.
- Validation: `read_amount()` and `read_pin()` in `utils.py` re-prompt or return `None` on bad input instead of letting an exception crash the program

9. Screenshots/Results

A sample console session: creating an account, depositing funds, checking the balance, viewing history, and logging out.

```
----- WELCOME TO THE ATM -----
1. Existing Account
2. New Account
3. Exit
Enter your choice: 1

Enter Account Number: 1001001
Enter PIN: 5511

PIN Correct! Welcome, Rusted Threads.

----- ATM MENU -----
Account: 1001001 | Holder: Rusted Threads
1. Check Balance
2. Deposit
3. Withdraw
4. Transaction History
5. Change Password
6. Exit
Enter your choice: 1
Balance = Rs. 3,000.00

----- ATM MENU -----
Account: 1001001 | Holder: Rusted Threads
1. Check Balance
2. Deposit
3. Withdraw
4. Transaction History
5. Change Password
6. Exit
Enter your choice: 2
Enter deposit amount: 3000
Deposit successful! Balance = Rs. 6,000.00

----- ATM MENU -----
Account: 1001001 | Holder: Rusted Threads
1. Check Balance
2. Deposit
3. Withdraw
4. Transaction History
5. Change Password
6. Exit
Enter your choice: 3
Enter withdrawal amount: 2000
Withdrawal successful! Balance = Rs. 4,000.00

----- ATM MENU -----
Account: 1001001 | Holder: Rusted Threads
1. Check Balance
2. Deposit
3. Withdraw
4. Transaction History
5. Change Password
6. Exit
Enter your choice: 4

--- Transaction History ---
[2026-09-26 02:24:25] Account created with balance Rs. 5,000.00
[2026-09-26 02:25:23] Deposited Rs. 300.00
[2026-09-26 02:25:53] Withdrew Rs. 2,000.00
[2026-09-26 02:29:41] Deposited Rs. 3,000.00
[2026-09-26 02:29:47] Withdrew Rs. 2,000.00

----- ATM MENU -----
Account: 1001001 | Holder: Rusted Threads
1. Check Balance
2. Deposit
3. Withdraw
4. Transaction History
5. Change Password
6. Exit
Enter your choice: 6
Thank you for using the ATM. Logging out...

----- WELCOME TO THE ATM -----
1. Existing Account
2. New Account
3. Exit
Enter your choice: 3
Goodbye!
```

Figure 7 – Sample Terminal Session

10. Testing Approach

The project includes an automated unittest suite of 29 tests across three files, run with:

```
python3 -m unittest discover -s test -v
```

- test_utils.py (13 tests) — PIN hashing consistency, currency formatting, and PIN-format validation.
- test_auth.py (9 tests) — account creation, incrementing account numbers, correct-PIN login, the 3-attempt lockout, rejecting a blocked account, unknown account numbers, and both correct/incorrect admin PIN paths.
- test_user_ops.py (7 tests) — deposit and withdrawal happy paths, rejecting negative/non-numeric amounts, rejecting an over-limit withdrawal, an exact-balance withdrawal, and both password-change paths.

Tests use unittest.mock.patch to simulate console input and to intercept save_db(), so the suite never touches the real atm_database.json and can be re-run safely at any time. All 29 tests pass

```
Ran 29 tests in 0.006s
OK
```

Manual/exploratory testing was also carried out end-to-end for the flows that are hardest to unit test in isolation: the full admin-forced-reset round trip, and the full admin-unblock round trip after a live lockout.

11. Challenges Faced

- Designing the lockout/reset interaction: making sure a force_reset account could not still be blocked, and that an admin unblock always also cleared the attempt counter, took a couple of passes to get the state transitions consistent.
- Testing input-driven functions: the original single-file script read input() directly inside deeply nested loops, which is hard to unit test. Splitting each menu action into its own function (Section 9) made it possible to mock input() and save_db() independently per test.
- Keeping the admin path and the user path symmetric: both needed the same kind of account-number lookup and formatting, which is why those pieces ended up in shared utils.py/database.py rather than duplicated in both admin_ops.py and user_ops.py.

12. Learnings & Key Takeaways

- Splitting a working single-file script into modules by responsibility (not just by size) makes the code far easier to unit test — the 29-test suite would have been much harder to write against the original monolithic version.
- A small persistence layer (database.py) that is the only place touching the storage format pays off immediately: every other module works with plain Python dictionaries and doesn't know or care that the data lives in JSON.
- Security-relevant logic (PIN hashing, lockout, forced reset) benefits from being centralised in one module (auth.py) rather than scattered, since it is the part of the system most worth reviewing carefully

13. Future Enhancements

- In-menu option for the admin to change their own PIN (currently requires editing the JSON file directly).
- Migrate storage from JSON to SQLite for safer concurrent access if the system were ever used from multiple terminals at once.
- Add configurable transaction limits (e.g. maximum daily withdrawal).
- Export a user's transaction history to CSV/PDF on request.
- A lightweight GUI (Tkinter) or web front-end reusing the same auth/user_ops/admin_ops modules unchanged.

14. References

- Python Software Foundation, hashlib — Secure hashes and message digests, Python 3 documentation.
- Python Software Foundation, json — JSON encoder and decoder, Python 3 documentation.
- Python Software Foundation, unittest.mock — mock object library, Python 3 documentation.
- VITyarthi, Build Your Own Project — General Project Instructions & Submission Guidelines (course project brief)



Thank You!